TEAM 3 - Whale Riders

DOCKER – SESSION REPORT

Container Lifecycle, Port Mapping, Nginx & Reverse Proxy

Session Date: 05 November 2026
Platform: AWS EC2
Operating System: Ubuntu

---

1. Introduction

In this session, we explored Docker container lifecycle management, port mapping, Nginx installation and reverse proxy configuration.

We used an Ubuntu EC2 instance on AWS to deploy multiple Docker containers .We then configured Nginx to act as a reverse proxy and route incoming browser requests to five different backend applications using different ports.

2. Objectives

The main objectives of this session were:

1. Understand the Docker container lifecycle.
2. Practise creating, starting, stopping, restarting, pausing and removing containers.
3. Understand Docker port mapping.
4. Install and configure Nginx on an Ubuntu EC2 instance.
5. Run multiple containers on different host ports.
6. Configure Nginx as a reverse proxy.
7. Access multiple applications through a single public IP address.
8. Identify and troubleshoot errors during implementation.

3. Environment Setup

We used the following environment for the practical implementation.

Component	Details
Cloud Platform	AWS
Service	Amazon EC2
Operating System	Ubuntu
Container Platform	Docker
Web Server	Nginx
Backend Applications	Apache HTTPD
Public Access	EC2 Public IPv4
Configuration	Nginx reverse proxy

Step 1: Connect to the EC2 instance

Connected to the Ubuntu EC2 instance using SSH.

Step 2: Verify Docker installation

```
None
sudo docker --version
sudo docker ps
```

These commands helped verify that Docker was installed and accessible.

4. Practical Implementation

TASK 1: Docker Container Lifecycle

4.1 Understanding Container Lifecycle

A Docker container goes through different states during its lifecycle.

Command	Purpose
docker create	Creates a container without starting it
docker start	Starts an existing container
docker stop	Stops a running container
docker restart	Restarts a container

docker pause	Temporarily freezes container processes
docker unpause	Resumes paused processes
docker rm	Removes a stopped container

Important concept:

`docker run` combines two operations:

- docker create
- docker start

4.2 Create a Container

Command:

```
None
sudo docker run -dit --name my-server ubuntu bash
```

Explanation:

- `-d`: Runs the container in the background.
- `-i`: Keeps standard input open.
- `-t`: Allocates a terminal.
- `--name my0-server`: Assigns a name to the container.
- `ubuntu`: Specifies the Docker image.
- `bash`: Runs the Bash shell.

4.3 Check Running Containers

```
None
sudo docker ps
```

This command displays currently running containers.

4.4 Stop the Container

```
None
sudo docker stop my-server
```

The container stops running but is not deleted.

4.5 View All Containers

```
None
sudo docker ps -a
```

This displays both running and stopped containers.

4.6 Start and Restart the Container

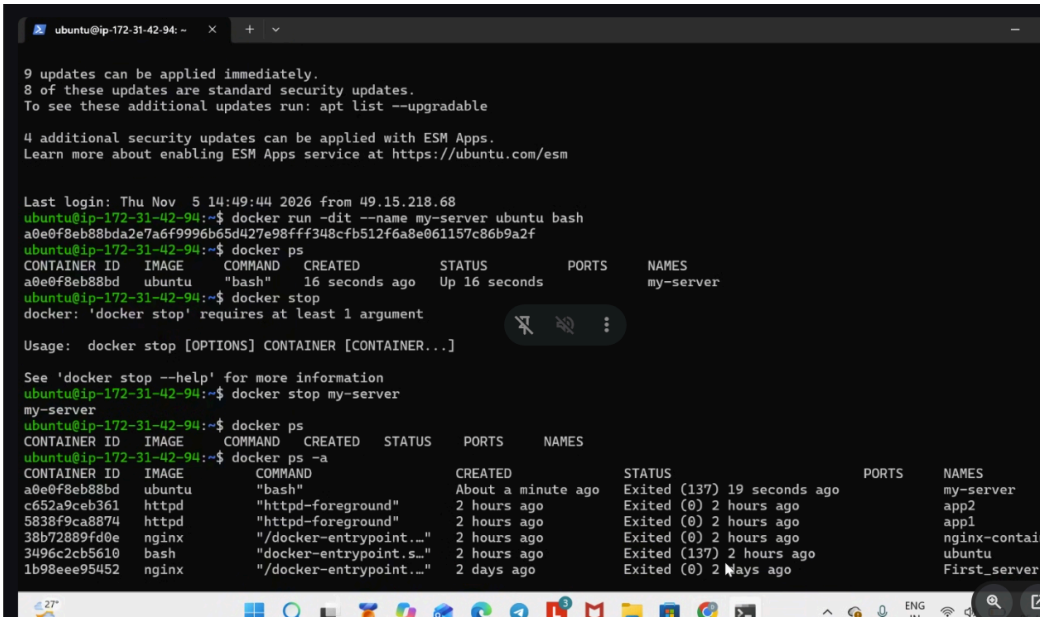
```
None
sudo docker start my-server
sudo docker restart my-server
sudo docker ps
```

4.7 Remove the Container

```
None
sudo docker stop my-server
sudo docker rm my-server
sudo docker ps -a
```

The container is removed permanently.

Screenshot 2 – Docker container lifecycle



```
ubuntu@ip-172-31-42-94:~$ docker start app1
app1
ubuntu@ip-172-31-42-94:~$ docker start app2
app2
ubuntu@ip-172-31-42-94:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED              STATUS              PORTS
NAMES
6ea739abcbc4   httpd     "httpd-foreground"       About a minute ago   Up About a minute   127.0.0.1:3005->80/tcp
app5
555301b4f21e   httpd     "httpd-foreground"       About a minute ago   Up About a minute   127.0.0.1:3004->80/tcp
app4
6aba00f02de1   httpd     "httpd-foreground"       2 minutes ago        Up 2 minutes        127.0.0.1:3003->80/tcp
app3
ef62d8496d9e   nginx     "/docker-entrypoint.…"   5 minutes ago        Up 5 minutes        0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
nginx_cont
ffff0e5e204e   nginx     "/docker-entrypoint.…"   9 minutes ago        Up 9 minutes        80/tcp
my-container
c652a9ceb361   httpd     "httpd-foreground"       2 hours ago          Up 3 seconds        127.0.0.1:3002->80/tcp
app2
5838f9ca8874   httpd     "httpd-foreground"       2 hours ago          Up 11 seconds       127.0.0.1:3001->80/tcp
app1
ubuntu@ip-172-31-42-94:~$
```

```
ubuntu@ip-172-31-42-94:~$ docker stop app2
app2
ubuntu@ip-172-31-42-94:~$ docker rm app1 app2
app1
app2
ubuntu@ip-172-31-42-94:~$ docker ps -a
CONTAINER ID   IMAGE         COMMAND                  CREATED          STATUS                     PORTS
      NAMES
6ea739abcbc4   httpd         "httpd-foreground"       21 minutes ago   Up 21 minutes              127.0.0.1:3005->80/tcp
      app5
555301b4f21e   httpd         "httpd-foreground"       21 minutes ago   Up 21 minutes              127.0.0.1:3004->80/tcp
      app4
6aba00f02de1   httpd         "httpd-foreground"       21 minutes ago   Up 21 minutes              127.0.0.1:3003->80/tcp
      app3
ef62d8496d9e   nginx         "/docker-entrypoint.…"   25 minutes ago   Up 25 minutes              0.0.0.0:8080->80/tcp, [::]:8080->80
/tcp   nginx_cont
ffff0e5e204e   nginx         "/docker-entrypoint.…"   29 minutes ago   Up 29 minutes              80/tcp
      my-container
38b72889fd0e   nginx         "/docker-entrypoint.…"   3 hours ago      Exited (0) 2 hours ago
      nginx-container
3496c2cb5610   bash          "docker-entrypoint.s…"   3 hours ago      Exited (137) 3 hours ago
      ubuntu
1b98eee95452   nginx         "/docker-entrypoint.…"   2 days ago       Exited (0) 2 days ago
      First_server
a0e9e2919c06   ubuntu        "bash"                   2 days ago       Exited (0) 2 days ago
      priceless_goldstine
ff22db5ea1e5   ubuntu        "/bin/bash"              2 days ago       Exited (0) 2 days ago
      suspicious_sinoussi
f3dcf4ddac2f   hello-world   "/hello"                 2 days ago       Exited (0) 2 days ago
      fervent_margulis
a9438465d838   hello-world   "/hello"                 2 days ago       Exited (0) 2 days ago
      nervous_chaum
ubuntu@ip-172-31-42-94:~$
```

```
ubuntu@ip-172-31-42-94: ~
ubuntu@ip-172-31-42-94:~$ docker stop my-server
my-server
ubuntu@ip-172-31-42-94:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
ubuntu@ip-172-31-42-94:~$ docker ps -a
CONTAINER ID   IMAGE         COMMAND                  CREATED             STATUS                      PORTS     NAMES
a0e0f8eb88bd   ubuntu        "bash"                   About a minute ago  Exited (137) 19 seconds ago           my-server
c652a9ceb361   httpd         "httpd-foreground"       2 hours ago         Exited (0) 2 hours ago            app2
5838f9ca8874   httpd         "httpd-foreground"       2 hours ago         Exited (0) 2 hours ago            app1
38b72889fd0e   nginx         "/docker-entrypoint.…"   2 hours ago         Exited (0) 2 hours ago            nginx-container
3496c2cb5610   bash          "docker-entrypoint.s…"   2 hours ago         Exited (137) 2 hours ago          ubuntu
1b98eee95452   nginx         "/docker-entrypoint.…"   2 days ago          Exited (0) 2 days ago             First_server
a0e9e2919c06   ubuntu        "bash"                   2 days ago          Exited (0) 2 days ago             priceless_goldstin
e
ff22db5ea1e5   ubuntu        "/bin/bash"              2 days ago          Exited (0) 2 days ago             suspicious_sinouss
i
f3dcf4ddac2f   hello-world   "/hello"                 2 days ago          Exited (0) 2 days ago             fervent_margulis
a9438465d838   hello-world   "/hello"                 2 days ago          Exited (0) 2 days ago             nervous_chaum
ubuntu@ip-172-31-42-94:~$ docker rm my-server
my-server
ubuntu@ip-172-31-42-94:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
ubuntu@ip-172-31-42-94:~$ docker ps -a
CONTAINER ID   IMAGE         COMMAND                  CREATED       STATUS                     PORTS     NAMES
c652a9ceb361   httpd         "httpd-foreground"       2 hours ago   Exited (0) 2 hours ago             app2
5838f9ca8874   httpd         "httpd-foreground"       2 hours ago   Exited (0) 2 hours ago             app1
38b72889fd0e   nginx         "/docker-entrypoint.…"   2 hours ago   Exited (0) 2 hours ago             nginx-container
3496c2cb5610   bash          "docker-entrypoint.s…"   2 hours ago   Exited (137) 2 hours ago           ubuntu
1b98eee95452   nginx         "/docker-entrypoint.…"   2 days ago    Exited (0) 2 days ago              First_server
a0e9e2919c06   ubuntu        "bash"                   2 days ago    Exited (0) 2 days ago              priceless_goldstine
ff22db5ea1e5   ubuntu        "/bin/bash"              2 days ago    Exited (0) 2 days ago              suspicious_sinoussi
f3dcf4ddac2f   hello-world   "/hello"                 2 days ago    Exited (0) 2 days ago              fervent_margulis
a9438465d838   hello-world   "/hello"                 2 days ago    Exited (0) 2 days ago              nervous_chaum
```

Result: Successfully practised the basic Docker container lifecycle commands and understood the difference between stopping and removing a container.

TASK 2: Nginx Installation on Ubuntu EC2

4.8 What is Nginx?

Nginx (pronounced Engine-X) is a high-performance web server that can also function as a reverse proxy, load balancer and HTTP cache.

In our practical session, we used Nginx to receive incoming HTTP requests and forward them to backend applications.

4.9 Install Nginx

Commands:

```
None
sudo apt update
sudo apt install nginx -y
nginx -v
```

Explanation:

- `apt update`: Updates the package lists.
- `apt install nginx -y`: Installs Nginx.
- `nginx -v`: Displays the installed Nginx version.

4.10 Verify Nginx Service

```
None
sudo systemctl status nginx
```

We checked whether the Nginx service was active and running.

To test the default webpage:

```
None
curl -s localhost | grep "<h1>"
```

Expected output:

```
None
<h1>Welcome to nginx!</h1>
```

4.11 Configure AWS Security Group

To access Nginx from a browser, we configured the EC2 security group's inbound rules.

Steps:

1. Open AWS Management Console.
2. Navigate to EC2.
3. Select the running instance.
4. Open the Security tab.
5. Click the associated Security Group.
6. Select Edit inbound rules.
7. Add an HTTP rule.

Type	Protocol	Port	Source
HTTP	TCP	80	My IP or 0.0.0.0/0

Save the inbound rules.

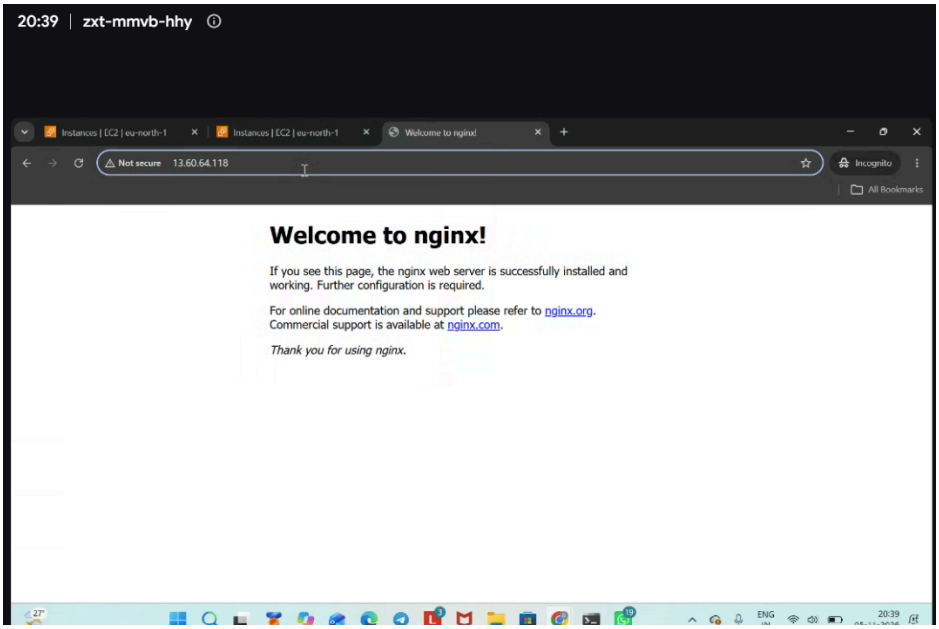
4.12 Access Nginx Through Browser

Open:

```
None
http://EC2_PUBLIC_IP
```

Expected result: Welcome to nginx!

Screenshot 4 – Default Nginx webpage



TASK 3: Nginx Inside Docker and Port Mapping

4.13 Stop Host Nginx

Before running Nginx inside Docker on another mapping, we stopped the Nginx service running directly on Ubuntu.

```
None
sudo systemctl stop nginx
curl -I localhost
```

We stopped the host Nginx service to avoid port conflicts on port 80.

4.14 Run Nginx Container

Command:

```
None
sudo docker run -d \
--name nginx-container \
-p 8080:80 \
nginx
```

Explanation:

Option	Purpose
-d	Runs the container in the background
--name	Assigns the container name
-p 8080:80	Maps host port 8080 to container port 80
nginx	Specifies the Nginx image

4.15 Understand Port Mapping

Port mapping connects a port on the EC2 host to a port inside the Docker container.

Format:

```
None
-p HOST_PORT:CONTAINER_PORT
```

Example:

```
None
-p 8080:80
```

Here:

- 8080 is the host port.
- 80 is the container port.

Request flow:

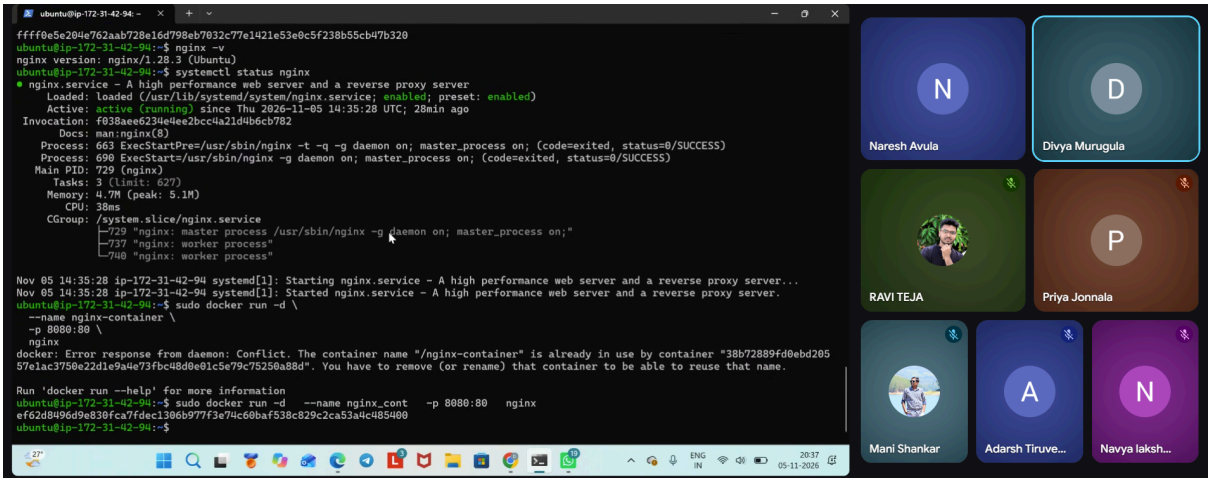
Browser → EC2 Public IP:8080 → Host Port 8080 → Container Port 80 → Nginx.

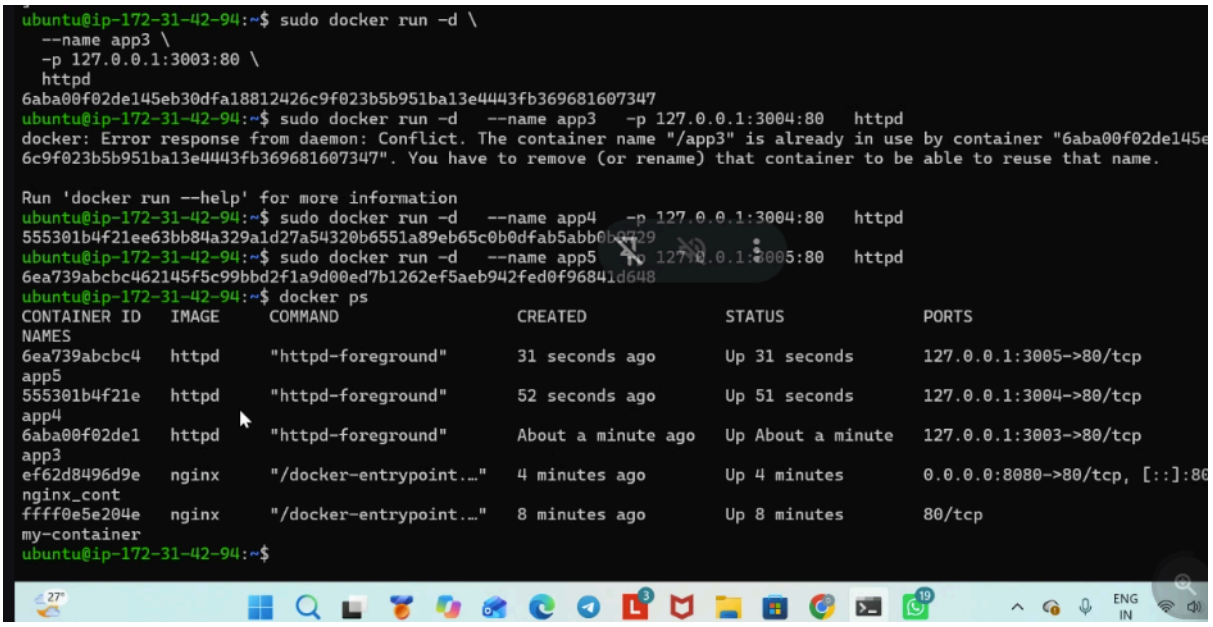
4.16 Verify the Container

```
None
sudo docker ps
sudo docker logs nginx-container
sudo docker inspect nginx-container
```

We checked whether the container was running and whether the port mapping was correctly configured.

Screenshot – Docker Nginx and port mapping





4.17 Configure Security Group

Added another inbound rule:

Type	Protocol	Port	Source
Custom TCP	TCP	8080	My IP

Access the application:

```
None
http://EC2_PUBLIC_IP:8080
```

TASK 4: Nginx as a Reverse Proxy

4.18 What is a Reverse Proxy?

A reverse proxy is a server that receives requests from clients and forwards those requests to the appropriate backend server.

In our implementation:

- Nginx acts as the reverse proxy.
- Apache HTTPD containers act as backend applications.
- Users access applications through the EC2 public IP.
- Nginx forwards requests based on URL paths.

4.19 Architecture of Our Implementation

We configured five backend applications using five different host ports.

Application	Docker Container	Host Port	Container Port	URL Path
Application 1	app1	3001	80	/app1/
Application 2	app2	3002	80	/app2/
Application 3	app3	3003	80	/app3/
Application 4	app4	3004	80	/app4/
Application 5	app5	3005	80	/app5/

4.20 Start Five Apache HTTPD Containers

We used the Apache HTTPD Docker image to create five different backend applications.

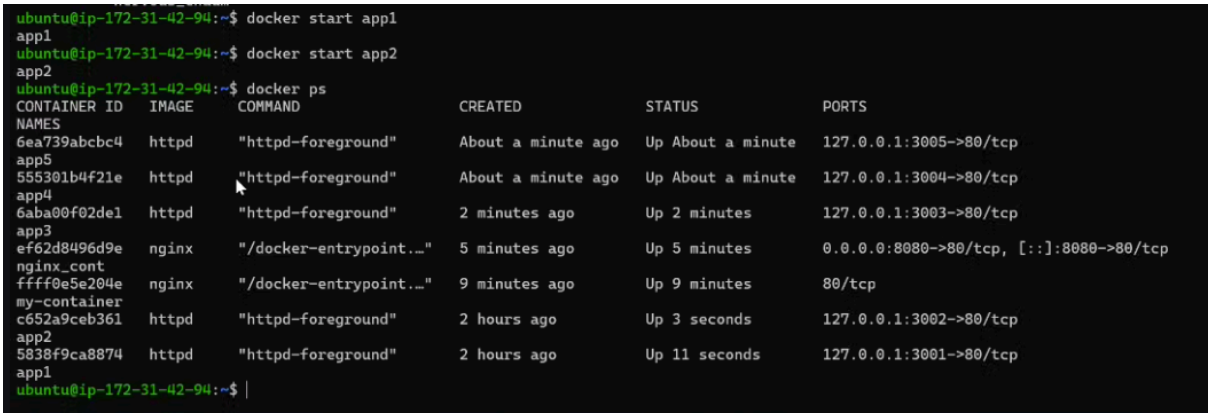
Application 1

```
None
sudo docker run -d \
--name app1 \
-p 127.0.0.1:3001:80 \
httpd
```

Application 2

None

```
sudo docker run -d \
--name app2 \
-p 127.0.0.1:3002:80 \
httpd
```



Application 3

None

```
sudo docker run -d \
--name app3 \
-p 127.0.0.1:3003:80 \
httpd
```

Application 4

None

```
sudo docker run -d \
--name app4 \
-p 127.0.0.1:3004:80 \
httpd
```

Application 5

None

```
sudo docker run -d \
--name app5 \
-p 127.0.0.1:3005:80 \
httpd
```

Verify all containers:

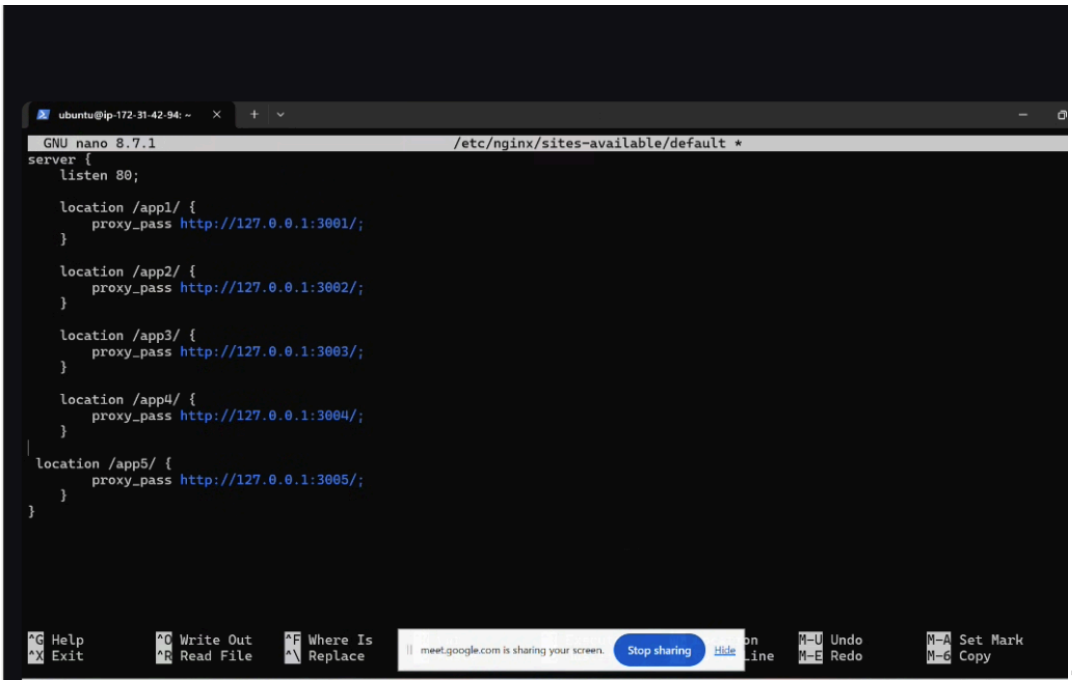
```
None
sudo docker ps
```

Understanding the command:

- `--name`: Gives each container a unique name.
- `-p`: Maps the host port to the container port.
- `127.0.0.1`: Makes the backend ports accessible only through the EC2 instance's localhost.
- `httpd`: Uses the Apache HTTPD image.

Each container listens on port 80 internally, but each has a different host port.

Screenshot – Five Docker containers



4.21

Create Different Webpages for Applications

We used Docker exec to execute commands inside each running container.

Application 1

```
None
sudo docker exec app1 sh -c 'echo "<h1>Welcome to Application 1</h1>" > /usr/local/apache2/htdocs/index.html'
```

Application 2

```
None
sudo docker exec app2 sh -c 'echo "<h1>Welcome to Application 2</h1>" > /usr/local/apache2/htdocs/index.html'
```

Application 3

```
None
sudo docker exec app3 sh -c 'echo "<h1>Welcome to Application 3</h1>" > /usr/local/apache2/htdocs/index.html'
```

Application 4

```
None
sudo docker exec app4 sh -c 'echo "<h1>Welcome to Application 4</h1>" > /usr/local/apache2/htdocs/index.html'
```

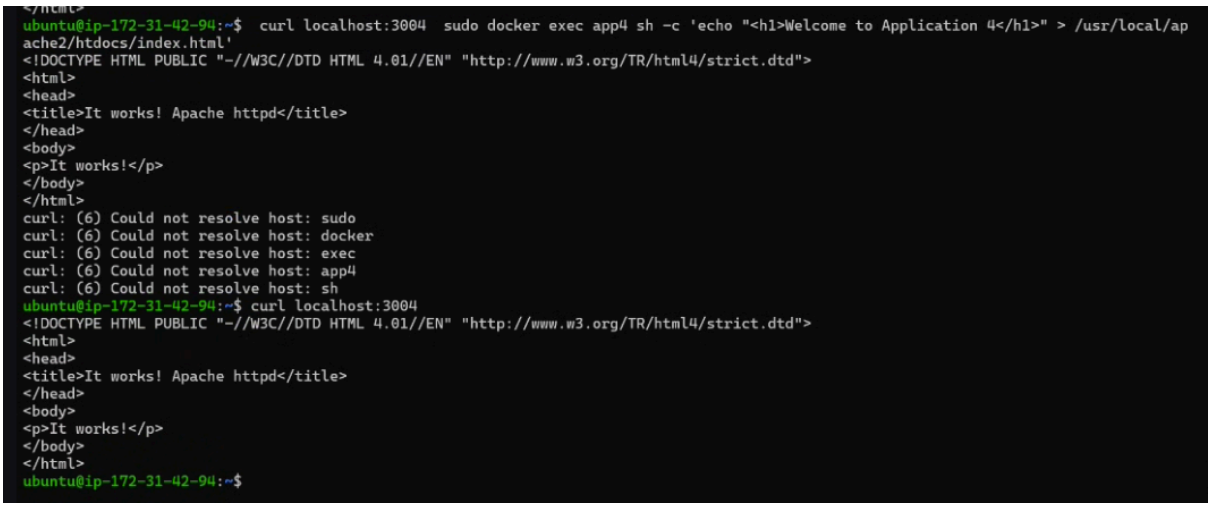
Application 5

```
None
sudo docker exec app5 sh -c 'echo "<h1>Welcome to Application 5</h1>" > /usr/local/apache2/htdocs/index.html'
```

Verify the backend applications:

```
None
curl localhost:3001
curl localhost:3002
curl localhost:3003
curl localhost:3004
curl localhost:3005
```





4.22 Start Host Nginx

After configuring the backend containers, we started Nginx on the Ubuntu EC2 host.

```
None
sudo systemctl start nginx
```

We created a backup of the existing Nginx configuration:

```
None
sudo cp /etc/nginx/sites-available/default
/etc/nginx/sites-available/default.bak
```

Then opened the default configuration file:

```
None
sudo nano /etc/nginx/sites-available/default
```

4.23 Configure Nginx Reverse Proxy

We configured Nginx to route requests to five backend applications.

Configuration:

None

```
server {
    listen 80;

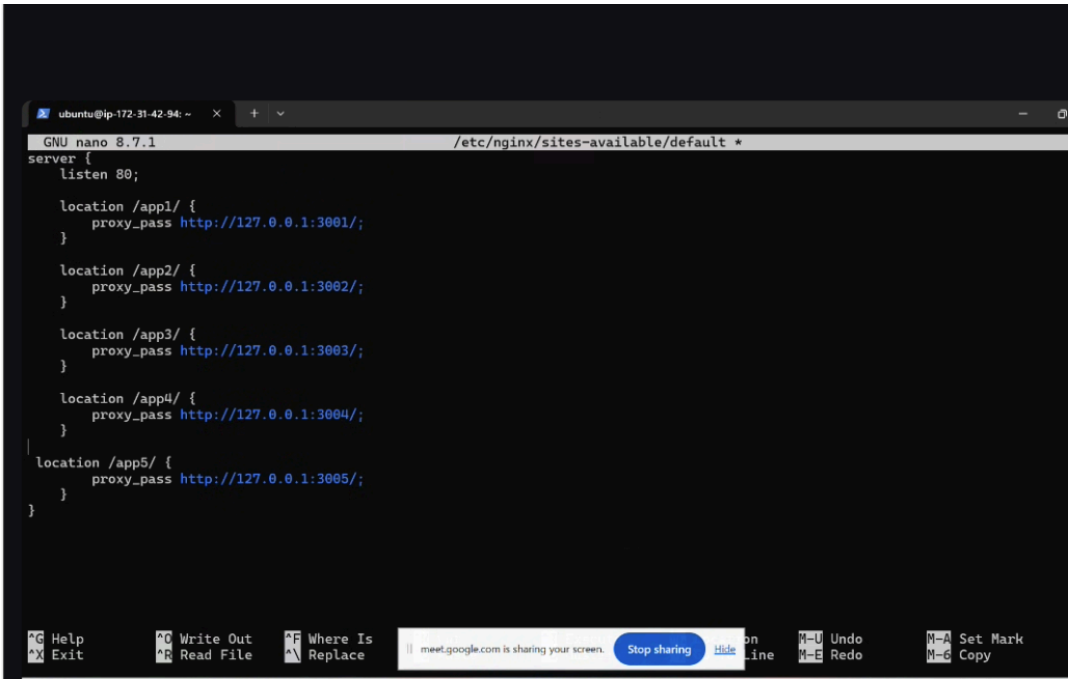
    location /app1/ {
        proxy_pass http://127.0.0.1:3001/;
    }

    location /app2/ {
        proxy_pass http://127.0.0.1:3002/;
    }

    location /app3/ {
        proxy_pass http://127.0.0.1:3003/;
    }

    location /app4/ {
        proxy_pass http://127.0.0.1:3004/;
    }

    location /app5/ {
        proxy_pass http://127.0.0.1:3005/;
    }
}
```



Explanation:

Configuration	Purpose
server { }	Defines the Nginx server block
listen 80;	Listens for incoming requests on port 80
location /app1/	Matches requests beginning with /app1/
proxy_pass	Forwards requests to the backend application
127.0.0.1:3001	Backend address for Application 1
127.0.0.1:3002	Backend address for Application 2
127.0.0.1:3003	Backend address for Application 3
127.0.0.1:3004	Backend address for Application 4
127.0.0.1:3005	Backend address for Application 5

Important: Both the location path and proxy_pass URL end with a trailing slash. Nginx removes the matching location prefix when forwarding the request.

Save and exit Nano:

- Ctrl + O
- Enter
- Ctrl + X

Screenshot 9 – Nginx reverse proxy configuration

[INSERT SCREENSHOT: Nano editor showing the complete five-application Nginx configuration]

Suggested filename: `09-nginx-reverse-proxy-config.png`

4.24 Test Nginx Configuration

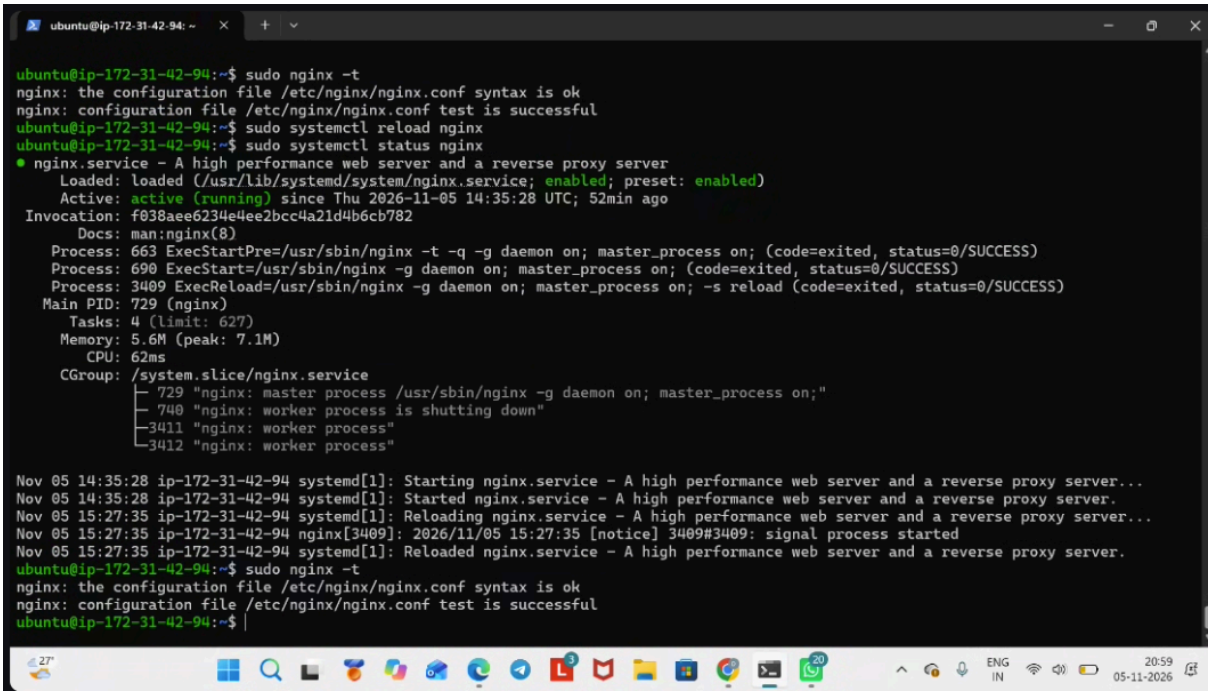
Before applying the configuration, we checked it for syntax errors.

```
None
sudo nginx -t
```

Expected output:

None

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is
successful
```





After successful validation, we reloaded Nginx.

None

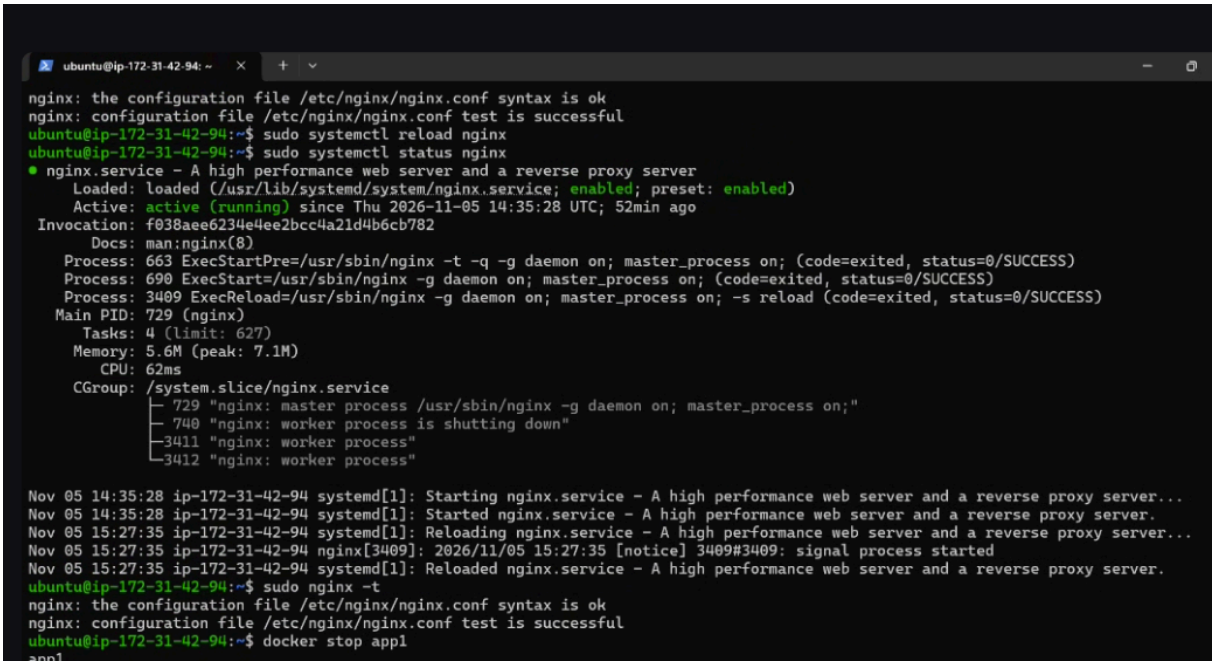
```
sudo systemctl reload nginx
```

Check the service:

None

```
sudo systemctl status nginx
```

We verified that the Nginx service was active and running.

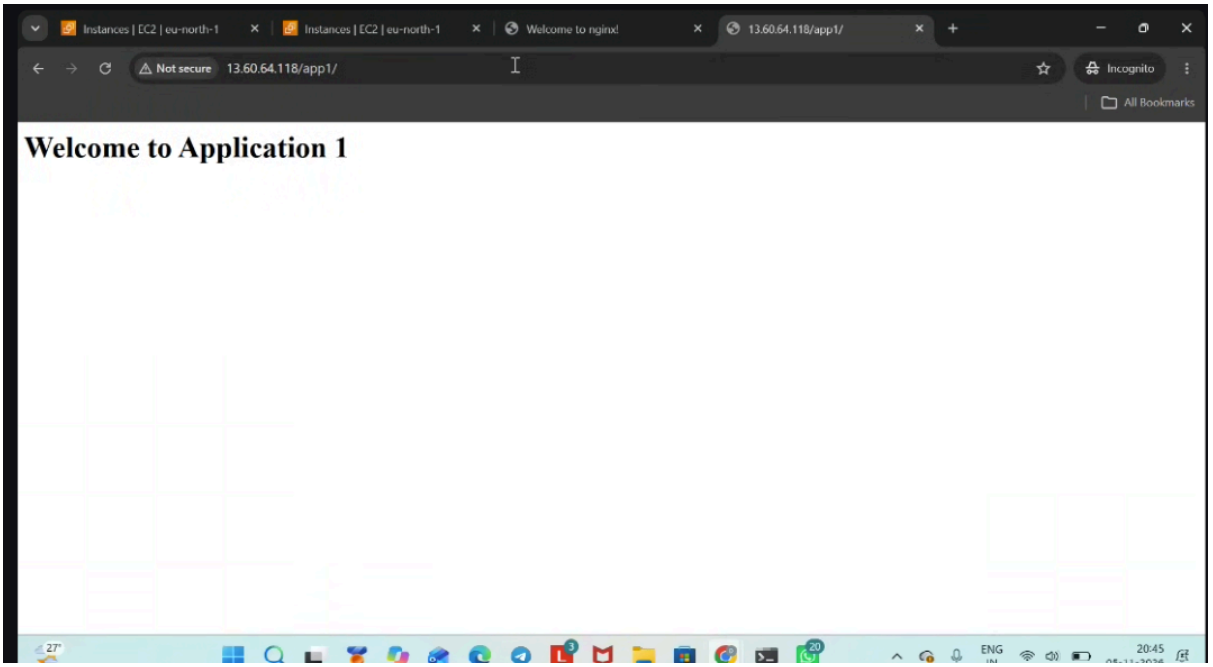


4.25 Access Five Applications Through Browser

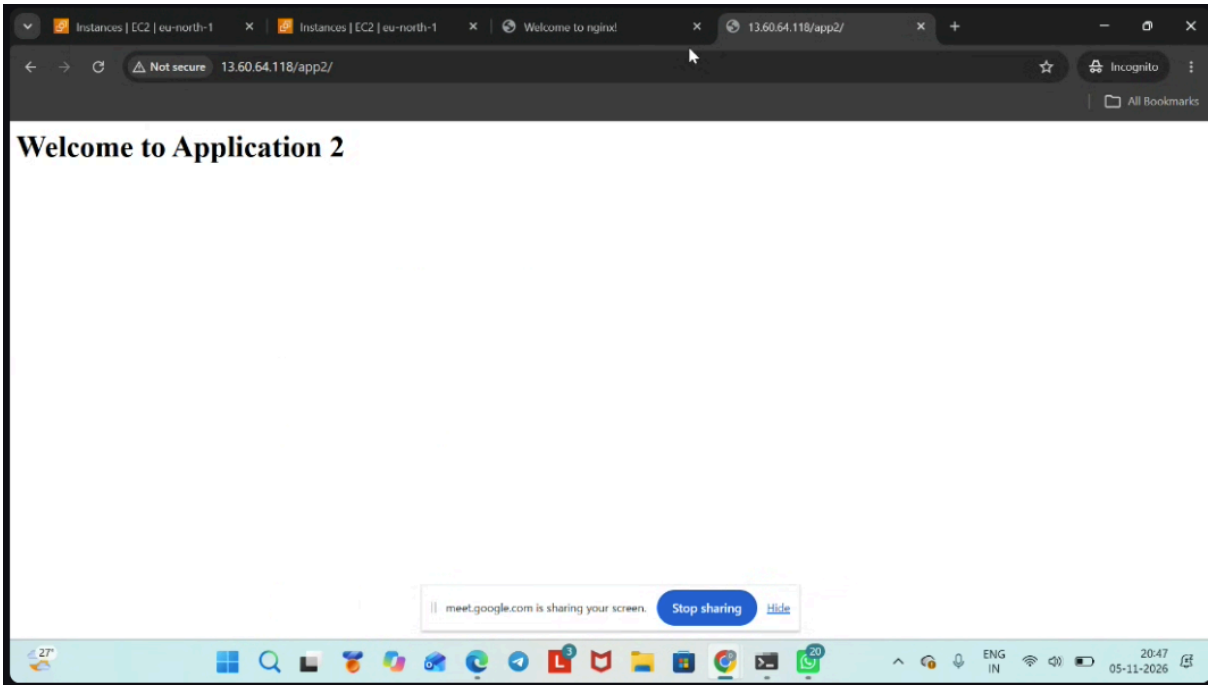
After configuring Nginx, we accessed the applications using the same EC2 public IP and different URL paths.

Application	Browser URL	Expected Output
Application 1	http://EC2_PUBLIC_IP/app1/	Welcome to Application 1
Application 2	http://EC2_PUBLIC_IP/app2/	Welcome to Application 2
Application 3	http://EC2_PUBLIC_IP/app3/	Welcome to Application 3
Application 4	http://EC2_PUBLIC_IP/app4/	Welcome to Application 4
Application 5	http://EC2_PUBLIC_IP/app5/	Welcome to Application 5

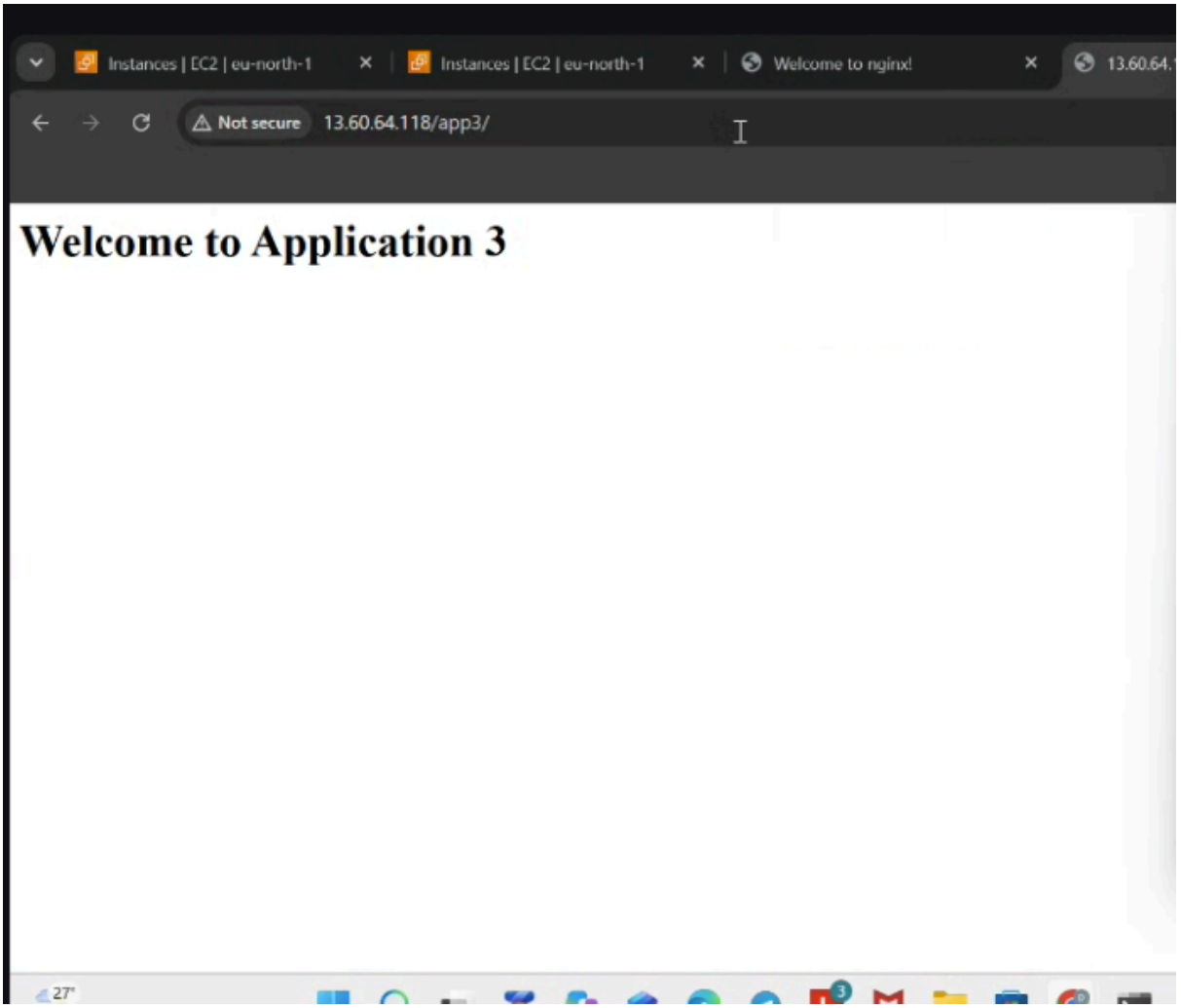
Screenshot – Application 1



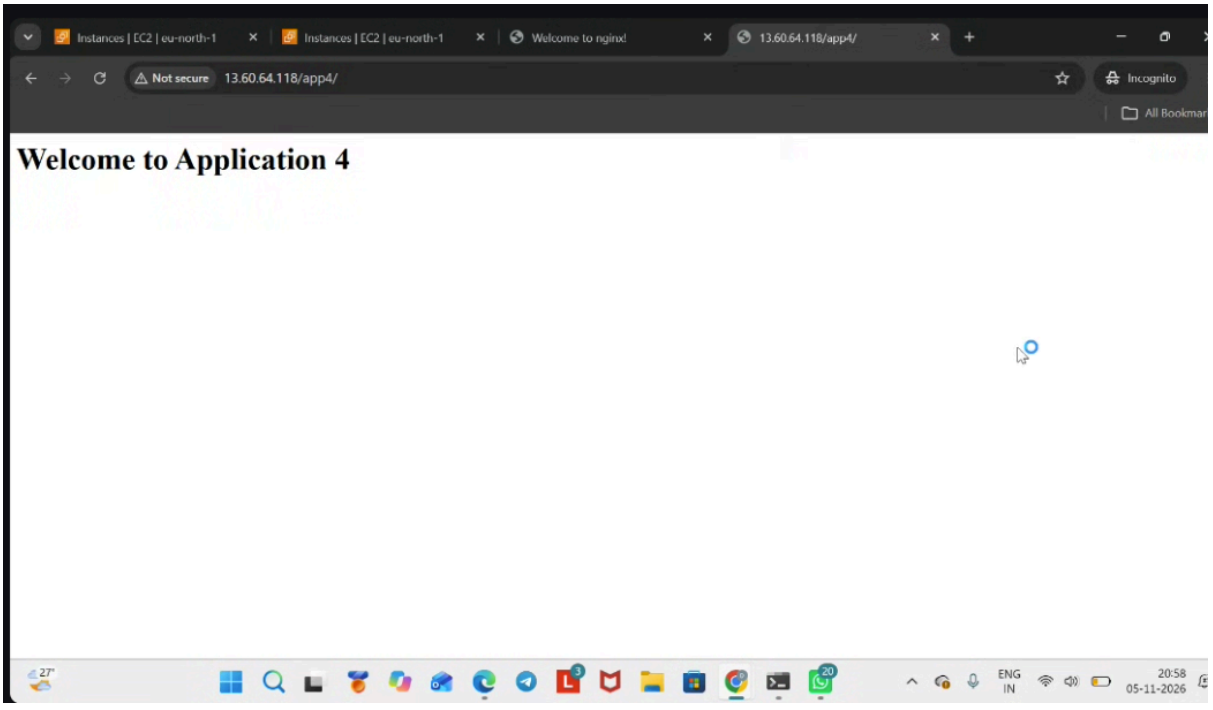
Screenshot – Application 2



Screenshot – Application 3



Screenshot – Application 4



5. Break-and-Fix Log

Issue 1: 404 Not Found While Accessing Application 3

Problem encountered:

During testing, when we accessed Application 3 through the browser using:

```
None
http://EC2_PUBLIC_IP/app3/
```

The browser displayed:

```
None
404 Not Found
nginx/1.28.3 (Ubuntu)
```

Troubleshooting performed:

1. Checked the Nginx reverse proxy configuration.
2. Verified the `/app3/` location block and its backend port mapping.
3. Tested the Nginx configuration using `sudo nginx -t`.
4. Confirmed that the configuration syntax was valid.
5. Reloaded the Nginx service to apply the configuration.
6. Tested the Application 3 URL again.

Commands used:

```
None
sudo nginx -t
sudo systemctl reload nginx
sudo systemctl status nginx
```

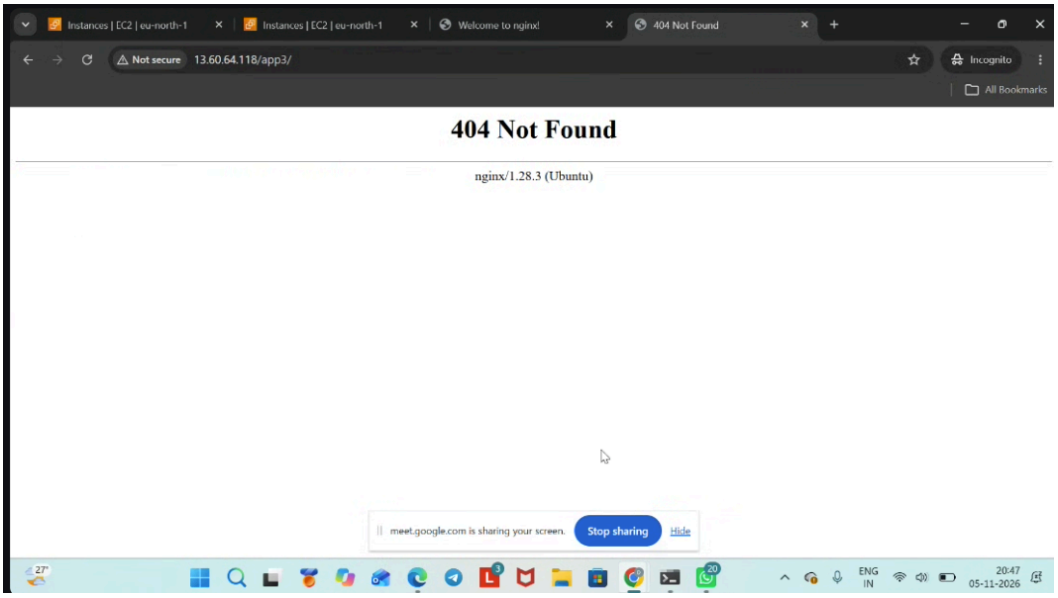
Final result:

After the configuration validation and reload, Application 3 was successfully displayed in the browser.

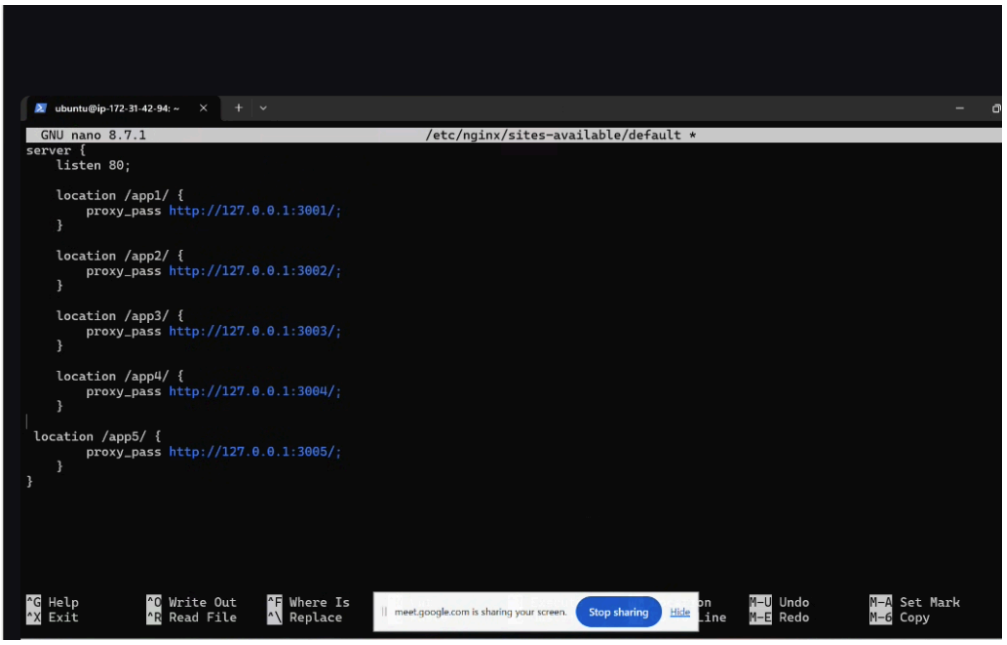
The final browser output showed:

```
None
Welcome to Application 3
```

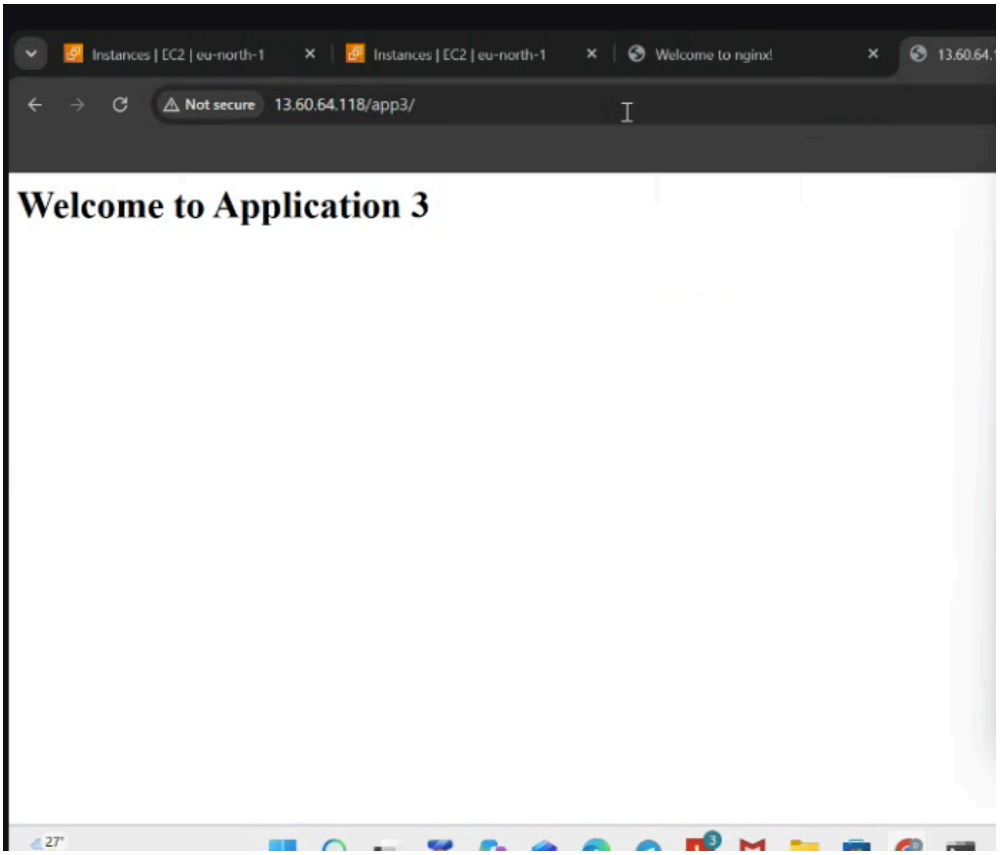
Root cause note: The initial screenshots show the 404 error followed by a successful response after configuration validation and reload. The exact initial cause was not conclusively established, so it should not be recorded as a confirmed syntax error.



Screenshot 18 – Troubleshooting and resolution



Successful result after fixing the issue



Troubleshooting Reference

Error	Possible reason	Troubleshooting
404 Not Found	Requested path does not match the active Nginx configuration or backend response	Check location block and URL
502 Bad Gateway	Backend container is unavailable	Check `sudo docker ps`
Connection refused	No service is listening on the requested port	Check service and container status
Browser timeout	Security group or network access issue	Check inbound rules
nginx -t failed	Configuration error	Check the reported file and line

6. Final Architecture

The final implementation follows this architecture:

Browser → EC2 Public IP → Nginx (Port 80) → Backend Docker Containers.

Request Path	Nginx Forwards To	Application
/app1/	localhost:3001	app1
/app2/	localhost:3002	app2
/app3/	localhost:3003	app3
/app4/	localhost:3004	app4
/app5/	localhost:3005	app5

Only port 80 needs to be publicly accessible for these reverse proxy routes. The backend ports are bound to localhost.

7. Final Verification

The following activities were performed during the practical session:

- Docker containers were created and managed using lifecycle commands.
- Nginx was installed on Ubuntu EC2.
- Nginx was also deployed inside a Docker container using port mapping.
- Five Apache HTTPD backend containers were configured on different host ports.
- Nginx was configured as a reverse proxy.
- Different application pages were accessed through URL paths.
- Nginx configuration was validated and the service was reloaded.
- The Application 3 404 issue was investigated and a successful browser response was obtained.

8. Key Learnings – Session Summary

1. Learned how to manage Docker containers using create, start, stop, restart, pause and remove commands.
2. Understood Docker port mapping and how host ports communicate with container ports.
3. Learned to install, configure and manage Nginx on an Ubuntu EC2 instance.
4. Implemented a reverse proxy to route requests to five Apache HTTPD containers through different URL paths.
5. Gained practical troubleshooting experience by identifying and resolving an Application 3 access issue.

9. Attendance and Team Roles

Team Member	Role	Contribution
J.S. Priya	Lead	Managed the session and tasks
Murugula Divya	Driver	Executed practical commands
Avula Naresh	Navigator	Verified commands and configurations
Raviteja(Matapathi)	Breaker	Identified and tested troubleshooting scenarios
M.Navya Lakshmi	Scribe	Prepared documentation and screenshots
Papana Mani Sankar	Presenter	Demonstrated the final implementation

10. GitHub Submission

All session commands, implementation notes, screenshots, troubleshooting details and learning outcomes are maintained in the team's GitHub repository.

GitHub Repository: https://github.com/Divya-Murugula/Whale_Riders

Folder: `day-01/`

Submission Date: 05 November 2026

Submitted By: Scriber

End of Day 1Session Report